

PCA & UMAP

Krishna Sekhar
Padma Venkatraman

Recap

From the PCA intro:

- PCA rotates to the data's **natural axes** — the *eigenvectors* $\vec{v}$ of the covariance C — ranked by variance (the eigenvalues λ)
- Each point gets a *score* on those axes ($z = \vec{v} \cdot \vec{x}$); a PC's *loadings* say *what* it is made of
- The one judgement call: how many PCs to keep

Today: the practical details (centering, scaling, choosing the number of PCs), where PCA *fails*, and a non-linear alternative — UMAP. (Full derivation: 01_b appendix + notebook 05.)

PCA — The Details

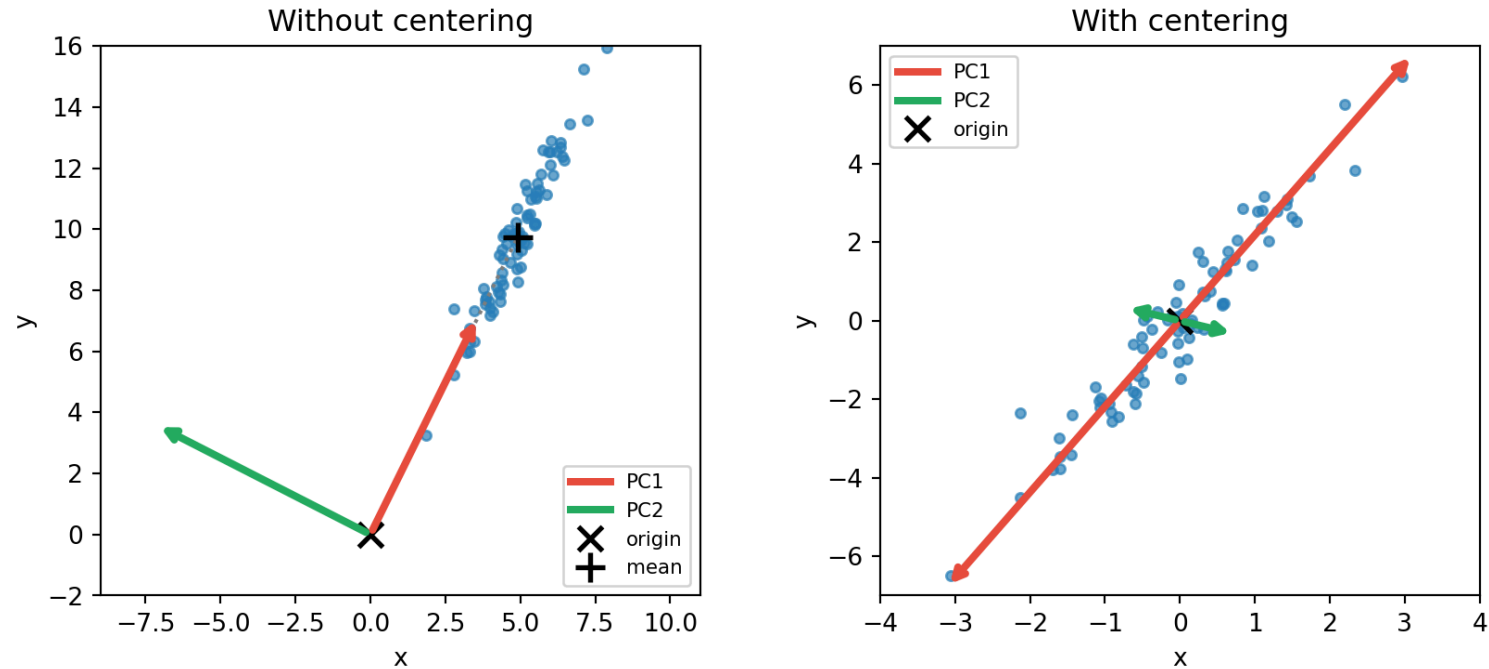
Why variance?

- If you measure a property of 1000 galaxies and they all have the same value, that tells you nothing — zero variance, zero information.
- High variance = data points actually *differ* along that direction, so it's potentially useful for analysis.
- PCA ranks directions by variance: “the directions where data points differ the most are the most informative”

Note

High variance doesn't always mean meaningful — it could be noise. But as a heuristic, it's a reasonable starting assumption.

Why centering matters



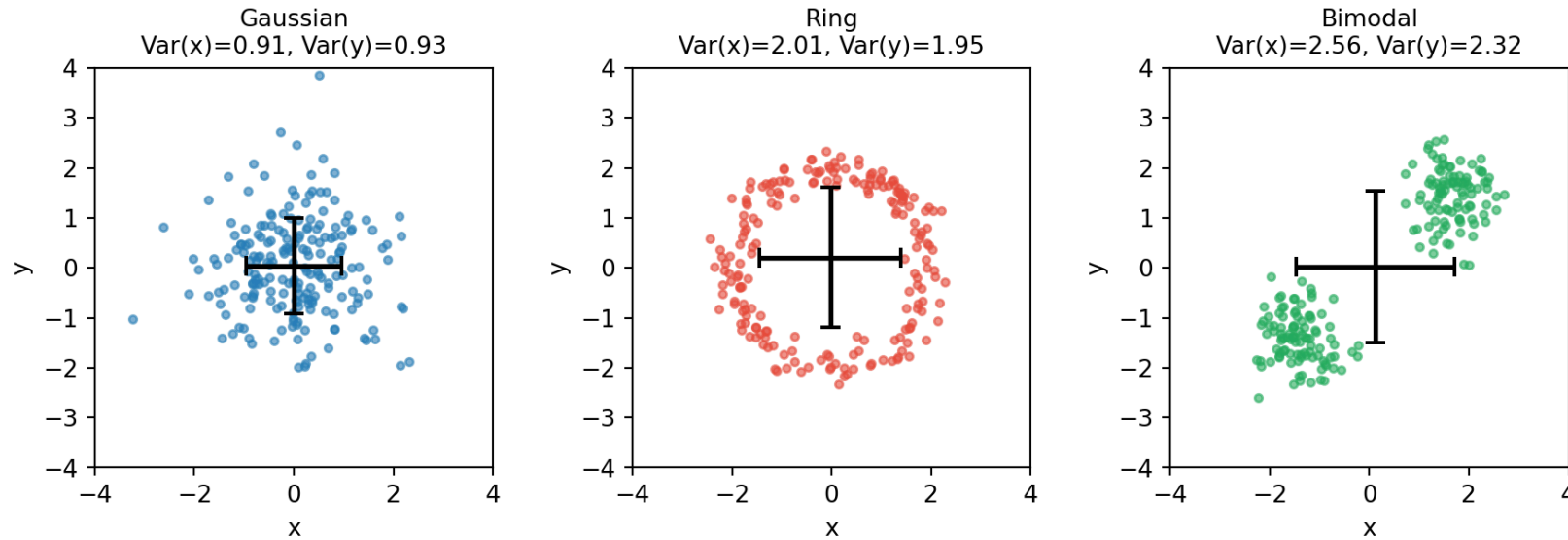
Without centering, PC1 points from the origin toward the mean (+) — not along the spread of the data.

The scaling gotcha

- Variables often have different units: flux in mJy (0.001–0.01), redshift (0.5–3.0), angular size in arcsec (1–100)
- Without standardising, variables with large numerical ranges would dominate PCA — not because they're more informative, but because of units

Fix: *standardise* each variable before PCA — the *z-score* transform $x \rightarrow \frac{x-\mu}{\sigma}$, where μ is the mean and σ is the standard deviation. After this, each variable has mean 0 and std 1, so PCA can compare the actual spread fairly.

Variance isn't always enough



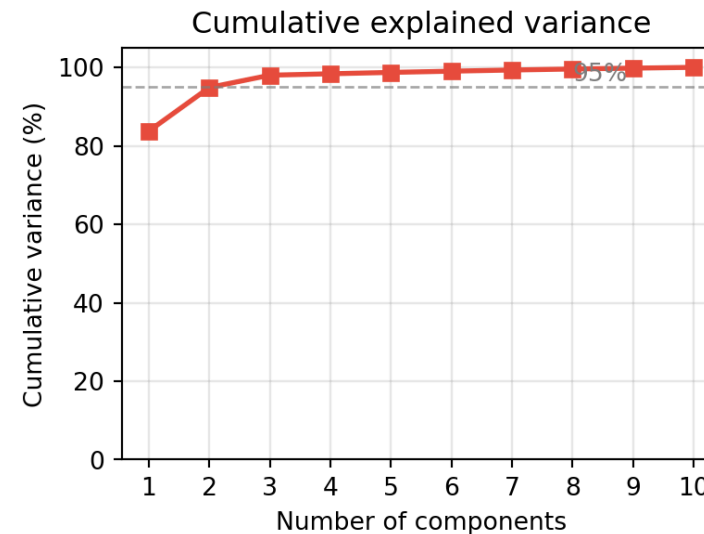
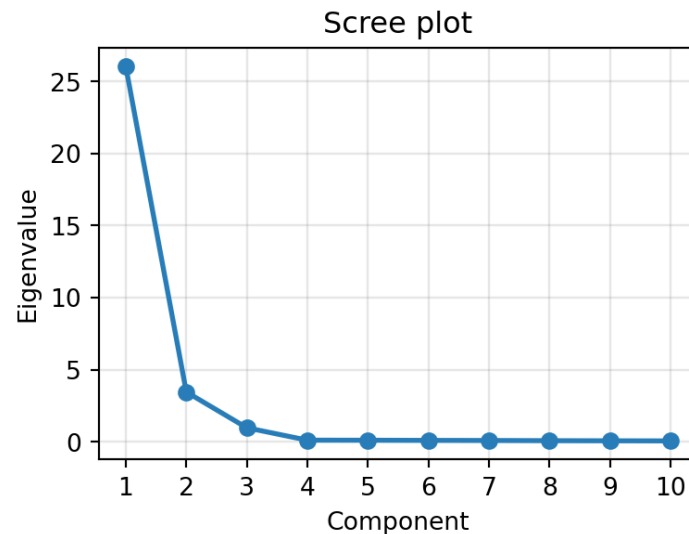
Similar variances in each dimension (crosshairs), very different structure. PCA only sees the variance.

Interpreting PCA Output

The scree plot

How do you decide how many PCs to keep? Plot the eigenvalues.

The *explained variance* is the fraction of total variance each PC captures: if eigenvalues are [25, 4, 1], PC1 explains $25/30 = 83\%$.



Choosing the number of components

Rules of thumb (none are absolute):

- **Elbow method** — look for where the scree plot flattens out. Components after the elbow are mostly noise.
- **Variance threshold** — keep enough PCs to explain 90–95% of total variance
- **Domain knowledge** — sometimes you know how many meaningful variables there should be

There is no single right answer. These are heuristics, not guarantees.

A worked example: FRI vs FR II

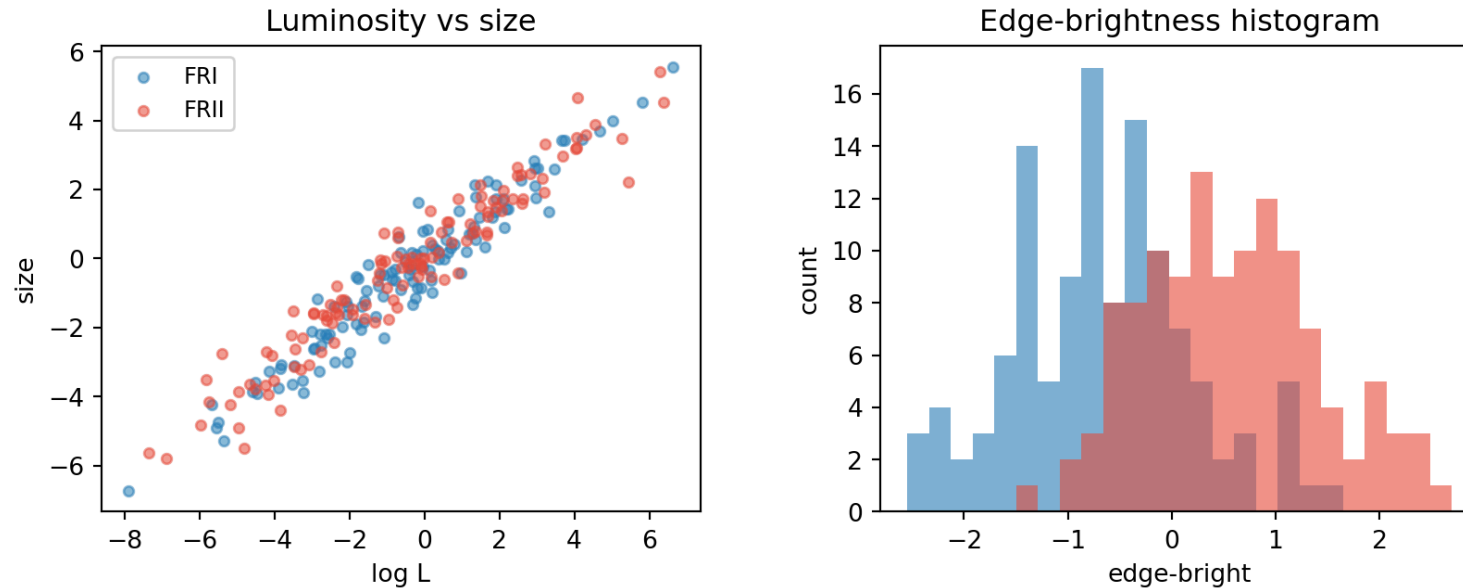
240 radio galaxies, two morphologies — **FRI** (edge-darkened) and **FR II** (edge-brightened) — described by seven measurements:

Feature	Measures	High in
log L	radio luminosity	— (scale)
size	projected linear size	— (scale)
edge-bright	edge / centre brightness	FR II
hotspot	hotspot prominence	FR II
lobe-collim	lobe collimation	FR II
jet-prom	jet prominence	FRI
core-dom	core dominance	FRI

No single one cleanly separates the classes — as the next slide shows.

Raw measurements overlap

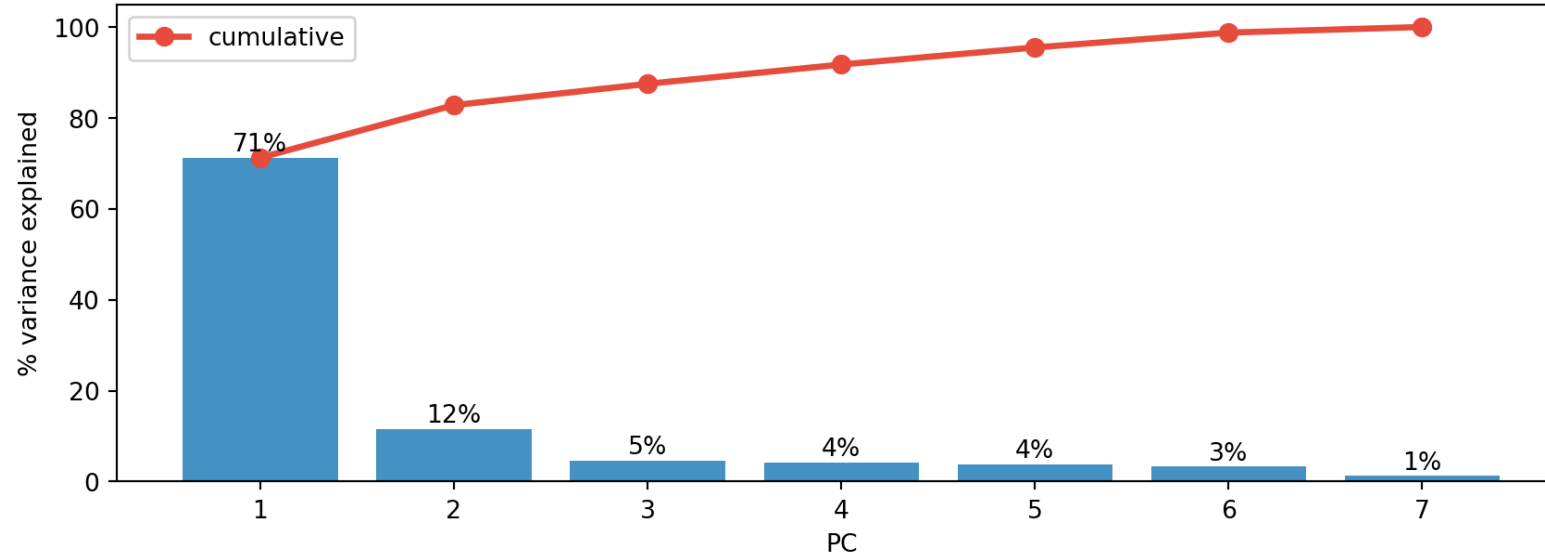
Plot the raw measurements: in every view the two classes sit on top of each other.



Left — the two *scale* features track each other, classes fully mixed. Right — the edge-brightness measurement shifts a little between classes but **still overlaps**.

The worked example: scree

Run PCA, look at the *scree plot* first.

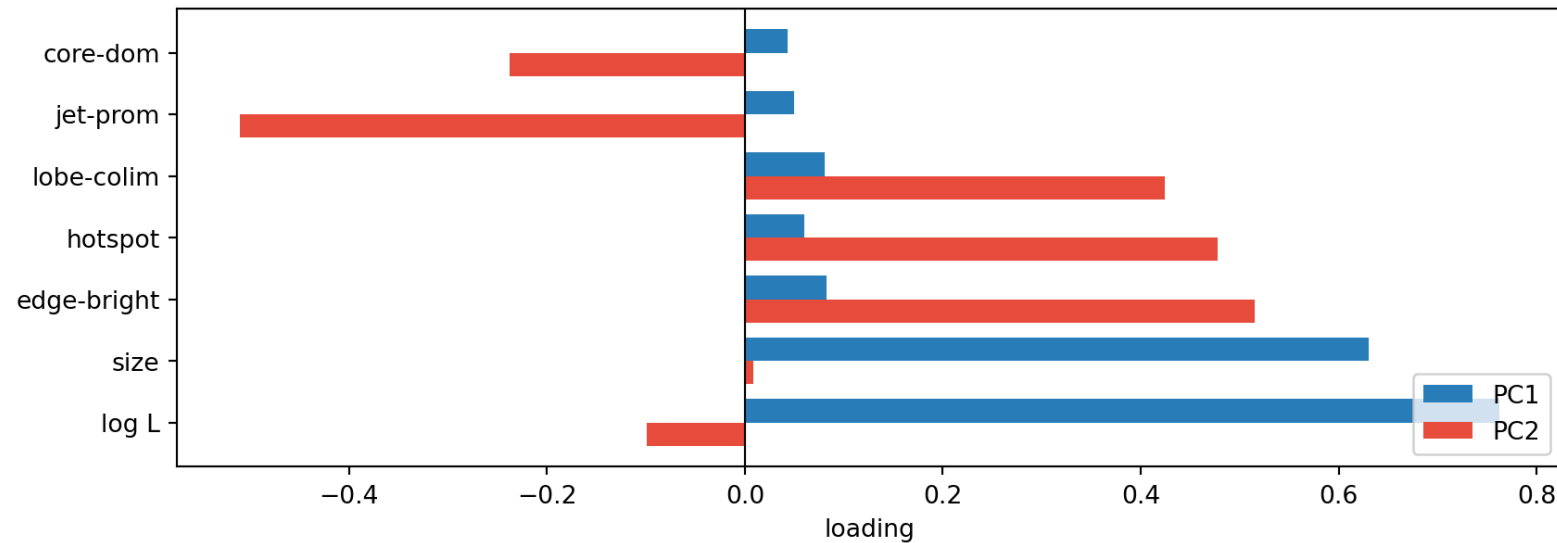


Discuss

Look at the scree plot. How many components would you keep — and on what basis?

The worked example: loadings

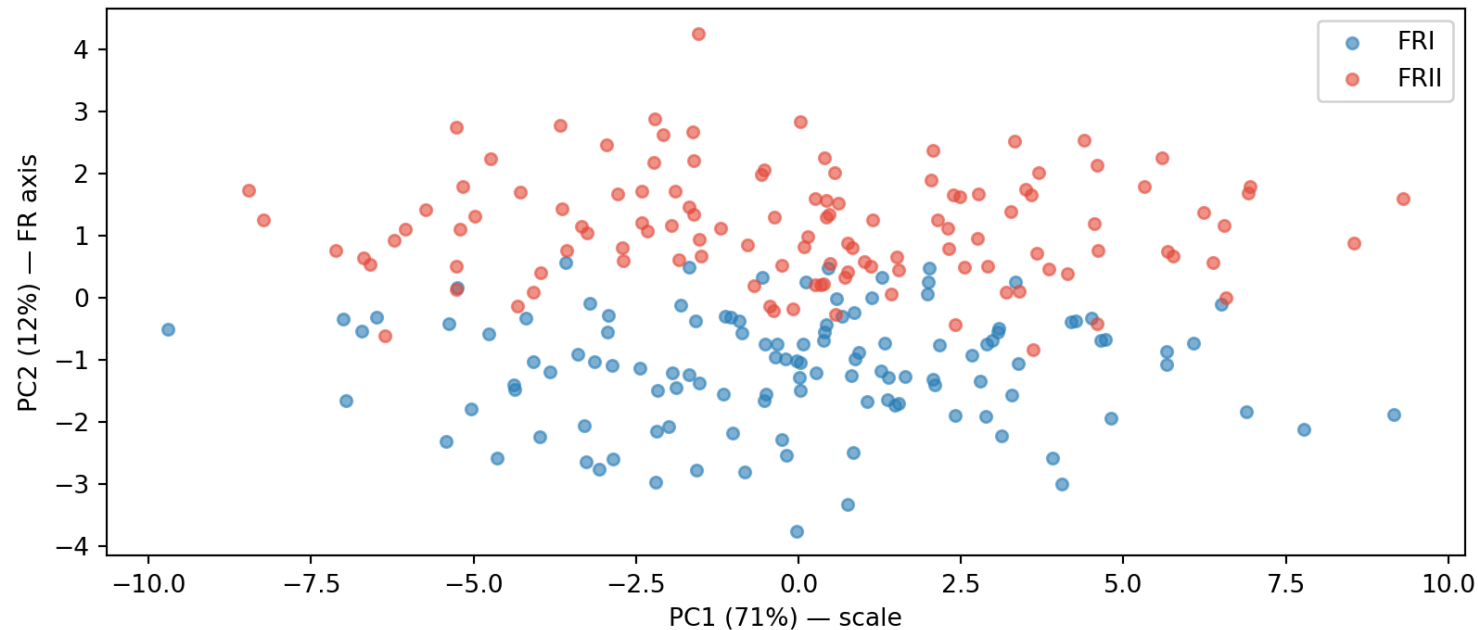
How many to keep is a variance call (scree → keep 2). The *loadings* then tell us *what those two PCs are*.



PC1 = overall **scale** (luminosity + size). *PC2* = **morphology contrast**: edge-brightening, hotspots & collimated lobes (+) vs prominent jets & dominant core (–) — the **FR signature**, a weighted blend of all five morphology features.

The worked example: structure appears

Project onto the top two PCs: $X_{\text{reduced}} = X_{\text{centered}} V_2$. This 2-column representation is what you use downstream.

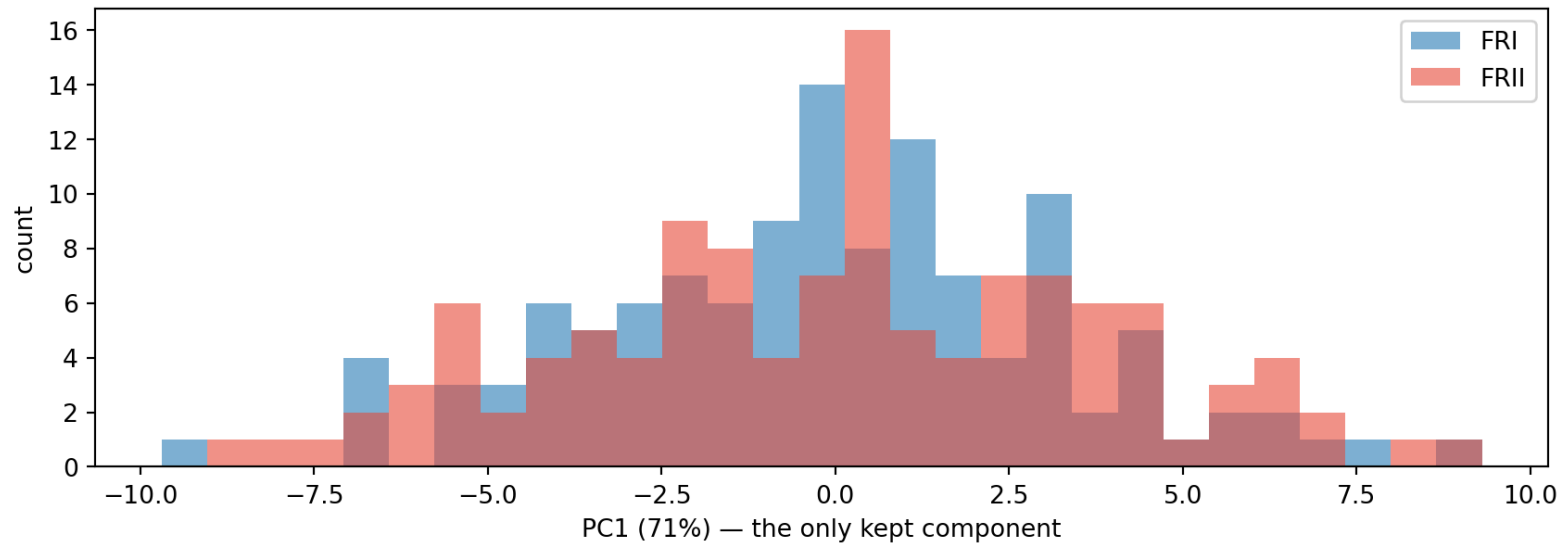


The classes separate cleanly — but **along PC2, not the high-variance PC1.**

PCA Failure Modes

Variance $\neq$ relevance

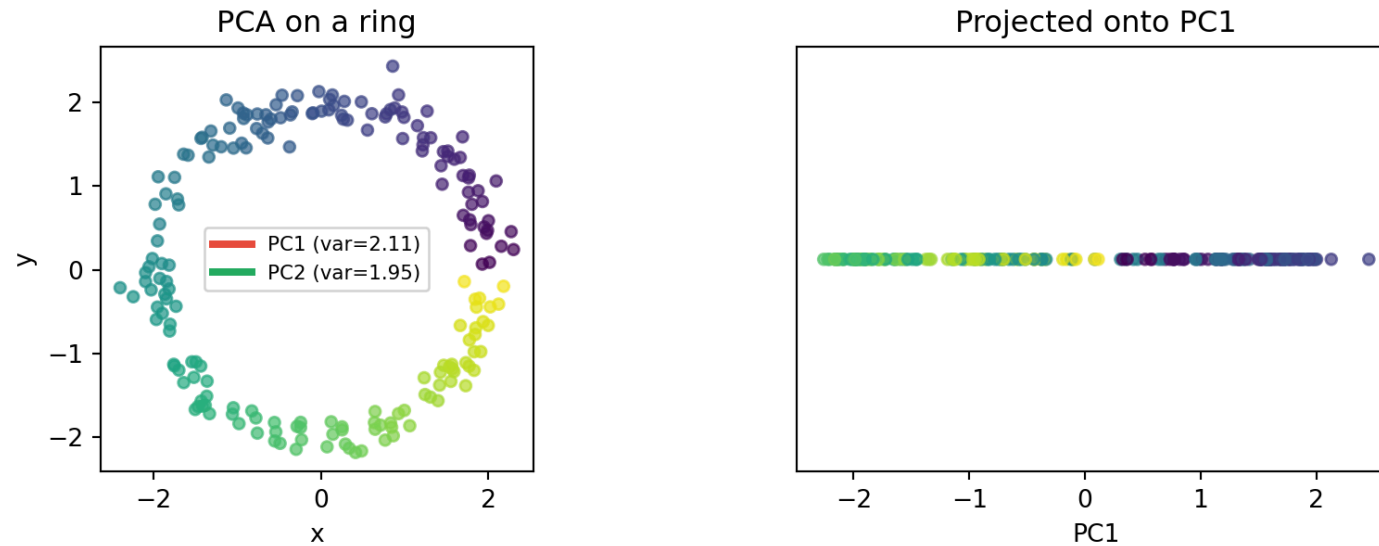
PCA ranks directions by *variance* — but the highest-variance direction need not be the one you care about. Back to FRI/FRII: keep only **PC1**, the biggest component.



FRI and FRII **overlap completely**. PC1 carried *scale*; the science lived in the smaller PC2 we threw away. High variance $\neq$ high relevance.

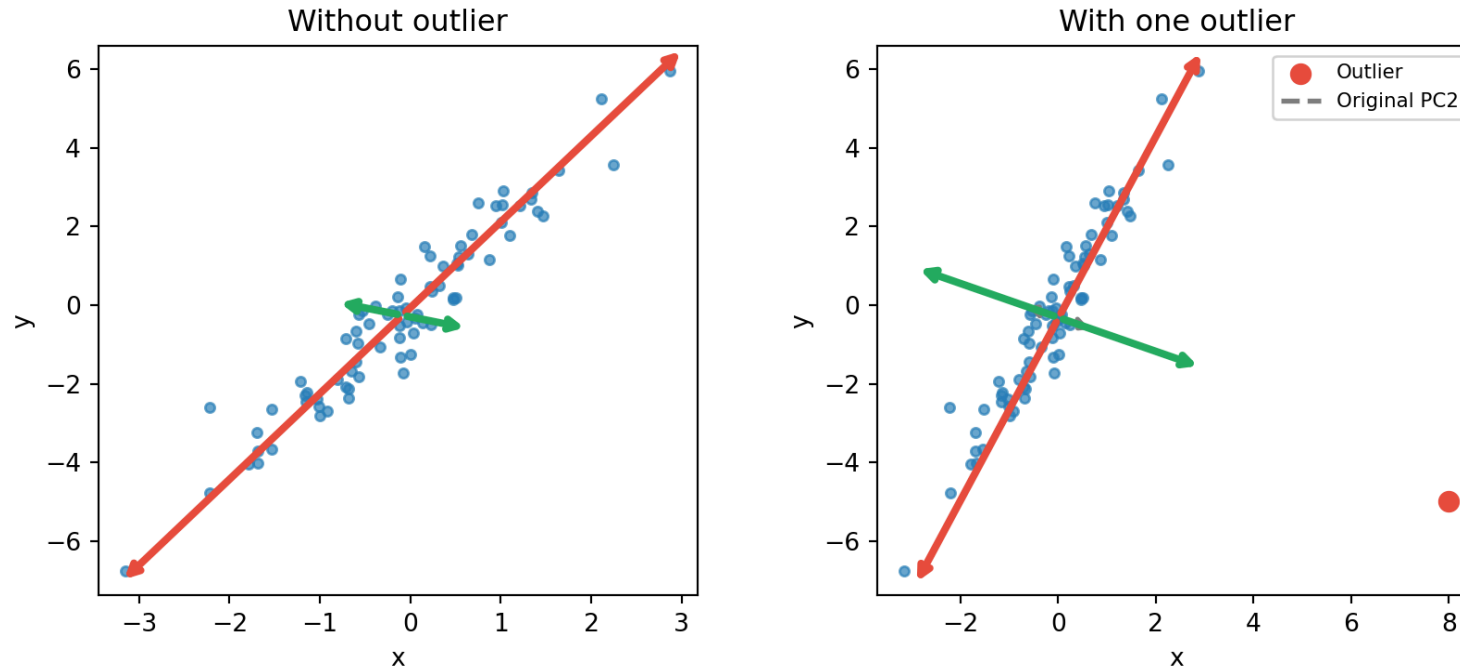
Non-linear structure

PCA assumes structure lies along **linear** directions. When data is non-linear, PCA's axes are meaningless.



Colour = position on the ring. Both PCs have similar variance — no reason to prefer either, and dropping either destroys the ring.

Outliers



A single extreme point rotates both PCs (compare green arrow to dashed original PC2).

When not to use PCA

- **Non-linear structure** — PCA assumes linearity. Rings, spirals, manifolds → use UMAP or t-SNE instead.
- **Unscaled features** — variables with large numerical ranges will dominate. Always standardise first.
- **All eigenvalues similar** — no clear low-rank structure. If every direction has similar variance, there's nothing to reduce.
- **Outlier-contaminated data** — clean or clip outliers before running PCA.

UMAP

Why UMAP?

- PCA fails on non-linear structure because it only looks at variance along straight lines
- We need a method that can preserve **non-linear** relationships
- UMAP (Uniform Manifold Approximation and Projection) does exactly this

UMAP — Intuition

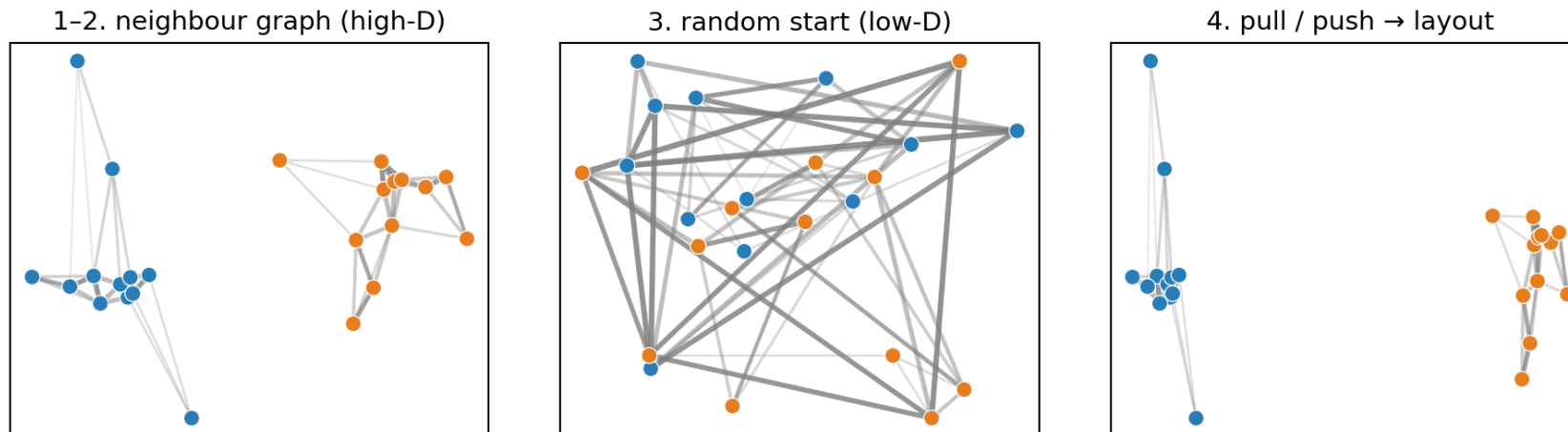
UMAP asks a different question than PCA:

- PCA: “which *directions* have the most variance?”
- UMAP: “which points are *neighbours*?”

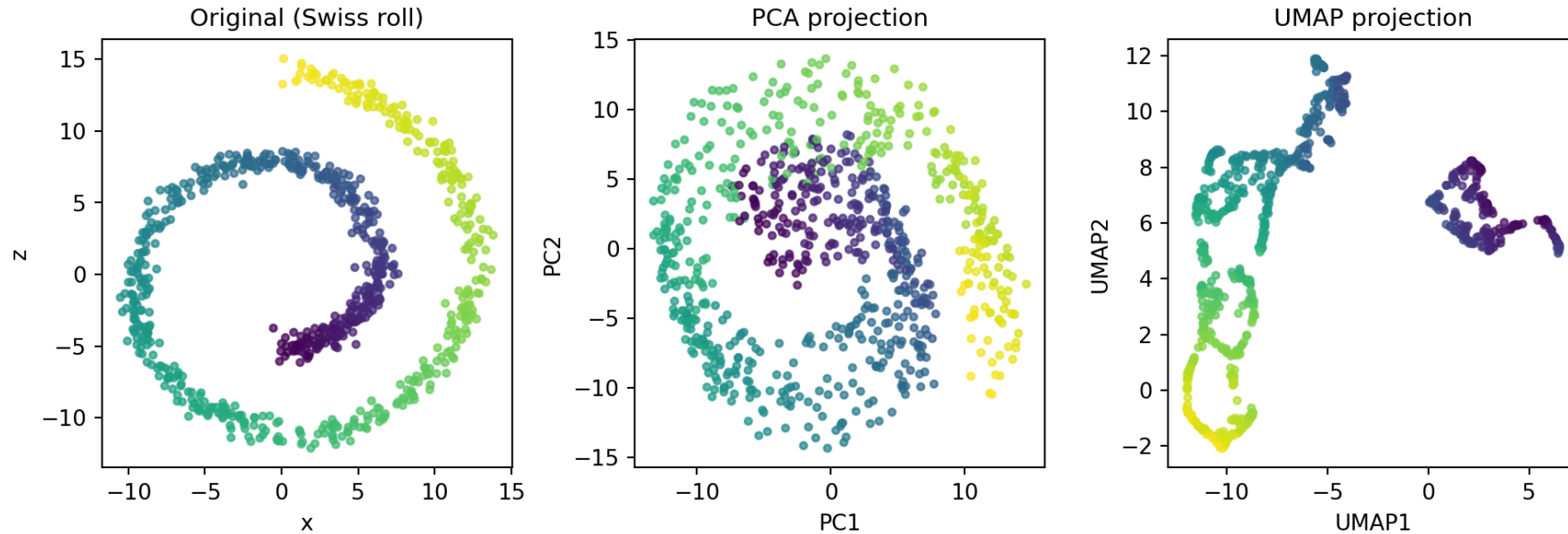
Why neighbours? Because the structure we care about — clusters, curves, manifolds — is defined by which points are close to which. If we can preserve those local relationships in a lower dimension, we preserve the structure, even if it's non-linear.

UMAP — The algorithm

1. For each point, find its k nearest neighbours in high-D.
2. Build a *weighted graph* — closer = stronger edge; each point gets its own distance scale (adaptive to local density).
3. Initialise a random layout in low-D (e.g. 2D).
4. Nudge positions so the graph matches: *pull* neighbours together, *push* non-neighbours apart.



PCA vs. UMAP — example



PCA flattens the Swiss roll — the colour gradient (the real structure) is scrambled. UMAP unrolls it.

PCA vs. UMAP

	PCA	UMAP
Structure	Linear	Non-linear
Preserves	Global variance	Local neighbourhoods
Axes meaningful?	Yes (eigenvectors)	No
Deterministic?	Yes	No (random init)
Speed	Fast	Slower
Scales to large n ?	Well	Reasonably

UMAP *hyperparameters*

A *hyperparameter* is a setting you choose *before* running the algorithm — it controls the algorithm's behaviour, but is not learned from the data.

Two key UMAP hyperparameters:

- **n_neighbors** — how many nearest neighbours to consider. Small → focus on very local structure (tight clusters). Large → capture more global structure (smoother layout).
- **min_dist** — how tightly points can pack in the low-D layout. Small → clumpy, separated clusters. Large → more spread out, preserves broader topology.

In code, that's all there is to it:

```
1 import umap
2 X_2d = umap.UMAP(n_neighbors=15, min_dist=0.1).fit_transform(X)
```

UMAP caveats

Things that are not meaningful in a UMAP plot:

- **Distances between clusters** — clusters that appear far apart are not necessarily more different than clusters that appear close
- **Cluster sizes** — a large cluster in UMAP is not necessarily more spread out in the original space
- **Results depend on hyperparameters** — different `n_neighbors` or `min_dist` can give qualitatively different layouts

UMAP is a visualisation and exploration tool. Don't over-interpret the geometry.

Summary

- PCA: powerful, fast, interpretable — but assumes linear structure. Always center (and usually standardise) first. Use scree plots to choose the number of components.
- UMAP: handles non-linear structure, great for visualisation — but axes are not interpretable and results depend on hyperparameters.
- Neither is universally better. Choose based on your data and your question.